

VB.NET Quick Revision Notes

1. Jagged Array

- Array of arrays.
- Each row can have different number of columns.

Example:

```
Dim A()() As Integer = {  
    New Integer() {1,2},  
    New Integer() {3,4,5}  
}
```

2. Dataset

- Part of ADO.NET.
 - Works on Disconnected Architecture.
 - Data remains available even after database connection is closed.
-

3. Interface and Inheritance

Interface

- Contains only declarations.
- Implemented using Implements keyword.

Interface ITest

Sub Show()

End Interface

Class Inheritance

- Achieved using Inherits keyword.

Class Child

Inherits Parent

End Class

4. File Handling

Namespace:

Imports System.IO

Classes:

- File
 - FileInfo
 - StreamReader
 - StreamWriter
-

5. Shadows Keyword

- Used to hide a member of the base class.

Public Shadows Sub Display()

6. ByRef

- Passes actual memory address.
- Changes made inside function affect original variable.

Sub Test(ByRef x As Integer)

7. Variable Declaration

Dim Age As Integer

DIM keyword is used to create variables.

8. Form Title Change

Me.Text = "Student Form"

Text property changes Form1 caption.

9. String Concatenation

Dim S As String

S = "VB" & ".NET"

& operator is used.

10. Label Control

- Used to display text.
- User cannot modify its content.

Property: Text

Method: Hide()

Event: Click

11. Five Important Controls

TextBox

Purpose: Input data

Property: Text

Method: Clear()

Event: TextChanged

Label

Purpose: Display text

Property: Text

Method: Hide()

Event: Click

Button

Purpose: Execute commands

Property: Text

Method: Focus()

Event: Click

ListBox

Purpose: Display list

Property: Items

Method: Add(), Remove()

Event: SelectedIndexChanged

ComboBox

Purpose: Drop-down list

Property: Items

Method: Add()

Event: SelectedIndexChanged

12. MsgBox()

Used to display messages.

MsgBox("Welcome")

13. InputBox()

Used to accept user input.

Dim Name As String

Name = InputBox("Enter Name")

Returns String value.

14. Procedure, Function and Event

Procedure (Sub)

Sub Display()

End Sub

- Performs task.
- Does not return value.

Function

Function Add() As Integer

- Returns value.

Event

Private Sub Button1_Click(...)

- Triggered by user/system action.
-

15. Decision Structures

If

If x > y Then

If...Else

If x > y Then

Else

End If

Elseif Ladder

If

Elseif

Else

End If

Select Case

Select Case Choice

Used for multiple conditions.

16. Loops

For Loop

For i = 1 To 10

Next

Known repetitions.

While Loop

While x < 10

End While

Checks condition first.

Do While Loop

Do While x < 10

Loop

Do Until Loop

Do Until x = 10

Loop

For Each Loop

For Each Item In Array

Next

Used for collections.

17. DateTimePicker (Calendar Control)

Purpose:

- Select Date and Time.

Property:

- Value

Event:

- ValueChanged

Example:

DateTimePicker1.Value

VERY IMPORTANT FOR EXAM

18. Overloading

Same method name + Different parameters.

Add(10,20)

Add(10,20,30)

Compile Time Polymorphism.

19. Overriding

Parent method redefined in child class.

Overridable Sub Show()

Overrides Sub Show()

Run Time Polymorphism.

20. Error vs Exception

Error

- Programming mistake.
- Compile time.

Example:

Dim x As Integer

x = "ABC"

Exception

- Runtime problem.

Example:

10 / 0

Handled using:

Try

Catch

Finally

End Try

21. Most Important One-Line Facts

- ✓ Dataset → Disconnected Architecture
- ✓ Interface → Implements
- ✓ Class Inheritance → Inherits
- ✓ File Handling → System.IO
- ✓ Variable Declaration → Dim
- ✓ String Concatenation → &
- ✓ Form Caption Change → Text Property
- ✓ Label → Display Text Only
- ✓ MsgBox → Output
- ✓ InputBox → Input

- ✓ Overloading → Same Name, Different Parameters
- ✓ Overriding → Redefine Parent Method
- ✓ MDI → Multiple Child Forms in One Parent Form
- ✓ SDI → One Document Per Window
- ✓ CLR → Common Language Runtime
- ✓ CTS → Common Type System
- ✓ FCL → Framework Class Library

BCA

(Sem-IV) — CC-9 (2021)

Object Oriented Programming using VB.NET

Time: 3 hours Full Marks: 50

Part - A

1. Multiple choice questions:

(i) VB.Net is a framework to develop:

(a) Software Application

(b) Web Application

(c) Android App

(d) All of these✓

(ii) Which function is used to concatenate all the specified elements of string array using the specified separator?

(a) Join()✓

(b) Concat()

(c) Both (a) and (b)

(d) None of these

(iii) Fixed values are also called _____

(a) Literals✓

(b) Global

(c) Dim

(d) None of these

(iv) Which one is not the component of Net Framework?

(a) The Net Framework Class Library

(b) Windows Forms

(c) ADO. Net

(d) C#✓

(v) Which reasons make VB.Net a widely used professional language?

(a) Structured language

(b) It can be compiled on a variety of Computer platforms

(c) Part of Net framework

(d) All of these✓

2. Fill in the blanks:

(a) The framework contains a large number of Class Library known as Framework Class Library (FCL).

(b) The Constant refers to fixed value that the program may not alter during its execution.

(c) An Exception in VB.Net refers to a problem that arises during program execution.

(d) When two or more processes execute simultaneously in a program, the process is known as Multithreading.

(e) CLR (Common Language Runtime) is the basic and virtual Machine Component of Net Framework.

Part - B

(Short-answer Type Questions)

3. Answer any four questions of the following:

(a) What is difference between Combo Box and List Box Control? Give examples.

ComboBox and **ListBox** are commonly used controls in VB.NET for displaying a list of items and allowing users to make selections. Although both controls display lists, they differ in appearance and functionality.

ComboBox Control

Definition

A **ComboBox** is a control that combines the features of a **TextBox** and a **ListBox**. It displays a drop-down list from which the user can select an item, and it can also allow the user to type a value.

Example

```
ComboBox1.Items.Add("BCA")
ComboBox1.Items.Add("BBA")
ComboBox1.Items.Add("B.Sc")
```

Output

Course ▼

BCA

BBA

B.Sc

Uses

- Selecting a course
- Choosing a city
- Selecting a country

ListBox Control

Definition

A **ListBox** is a control that displays a list of items continuously. Users can select one or more items directly from the visible list.

Example

```
ListBox1.Items.Add("Apple")
ListBox1.Items.Add("Mango")
ListBox1.Items.Add("Orange")
```

Output

Apple

Mango

Orange

Uses

- Displaying item lists
- Subject selection
- Product lists

(b) What is Variable? How variable is declared in VB.Net? Give proper example with program.

A **variable** is a named memory location used to store data temporarily during the execution of a program. The value stored in a variable can change during program execution.

Variables are used to store different types of data such as numbers, characters, strings, dates, and logical values.

Definition of Variable

A variable is an identifier that holds a value which can be modified during program execution.

Example

```
Dim Age As Integer = 20
```

Here:

- **Age** is the variable name.
 - **Integer** is the data type.
 - **20** is the value stored in the variable.
-

Variable Declaration in VB.NET

Variables are declared using the **Dim** keyword.

Syntax

```
Dim VariableName As DataType
```

Example

```
Dim Name As String  
Dim Marks As Integer  
Dim Salary As Double
```

Initialization of Variables

A variable can be declared and initialized at the same time.

Example

Dim Name As String = "Rahul"

Dim Marks As Integer = 85

Common Data Types Used in Variables

Data Type Description		Example
Integer	Stores whole numbers	100
Double	Stores decimal numbers	25.75
String	Stores text	"VB.NET"
Boolean	Stores True or False	True
Date	Stores date values	#01/01/2025#

Example Program

```
Public Class Form1

    Private Sub Button1_Click(sender As Object,
        e As EventArgs) Handles Button1.Click

        Dim Name As String = "Rahul"
        Dim Age As Integer = 20
        Dim Marks As Double = 85.5

        MsgBox("Name : " & Name & vbCrLf &
            "Age : " & Age & vbCrLf &
            "Marks : " & Marks)

    End Sub

End Class
```

Output

Name : Rahul

Age : 20

Marks : 85.5

Rules for Naming Variables

1. Variable name must begin with a letter.
2. Spaces are not allowed.
3. Special characters are not allowed except underscore (_).
4. Reserved keywords cannot be used as variable names.
5. Variable names should be meaningful.

Advantages of Variables

1. Store data temporarily.
2. Make programs flexible and reusable.
3. Simplify calculations.
4. Improve readability of programs.
5. Facilitate data manipulation.

(c) What is SDI and MDI? Explain it

SDI and MDI are two types of user interface designs used in VB.NET applications for managing windows and documents. They determine how users interact with multiple forms or documents within an application.

SDI (Single Document Interface)

Definition

SDI (Single Document Interface) is an interface in which each document or form is opened in a separate window. Every window operates independently of the others.

In SDI applications, only one document is associated with one window at a time.

Characteristics of SDI

1. Each document has its own separate window.
2. Simple and easy to develop.
3. Easy to manage and maintain.
4. Uses fewer system resources.
5. Suitable for small applications.

Advantages of SDI

1. Simple user interface.
2. Easy navigation.
3. Less memory consumption.
4. Easier programming and maintenance.

Disadvantages of SDI

1. Multiple documents create multiple windows on the desktop.
2. Difficult to manage many open documents.
3. Less efficient for document-based applications.

Examples

- Notepad
 - Paint
 - Calculator
-

MDI (Multiple Document Interface)

Definition

MDI (Multiple Document Interface) is an interface in which multiple child windows (documents) are opened and managed inside a single parent window.

The parent form acts as a container for all child forms.

Characteristics of MDI

1. One parent form contains multiple child forms.
2. Child forms remain inside the parent window.
3. Shared menus and toolbars can be used.
4. Better organization of multiple documents.
5. Suitable for large applications.

Advantages of MDI

1. Multiple documents can be managed in a single window.
2. Better workspace organization.
3. Shared menus and toolbars reduce duplication.
4. Improves user productivity.

Disadvantages of MDI

1. More complex to design and develop.
2. Consumes more memory and resources.
3. Child windows may become cluttered.
4. Navigation becomes difficult when many child forms are open.

Examples

- Visual Studio
- Adobe Photoshop (older versions)
- Database Management Applications

(d) Write a program to input three numbers and display the largest number. (Design the proper interface using controls)

LARGEST NUMBER PROGRAM

First Number : [TextBox1]

Second Number : [TextBox2]

Third Number : [TextBox3]

Largest Number : [TextBox4]

[FIND] [CLEAR]

Control	Name
---------	------

Label	Label1, Label2, Label3, Label4
-------	--------------------------------

TextBox	TextBox1
---------	----------

TextBox	TextBox2
---------	----------

TextBox	TextBox3
---------	----------

TextBox	TextBox4
---------	----------

Button	Button1 (Find)
--------	----------------

Button	Button2 (Clear)
--------	-----------------

```
Private Sub Button1_Click(sender As Object,  
e As EventArgs) Handles Button1.Click
```

```
    Dim A, B, C As Integer
```

```
    A = Val(TextBox1.Text)
```

```
    B = Val(TextBox2.Text)
```

```
    C = Val(TextBox3.Text)
```

```
    If A >= B And A >= C Then
```

```
        TextBox4.Text = A.ToString()
```

```
    ElseIf B >= A And B >= C Then
```

```
        TextBox4.Text = B.ToString()
```

```
    Else
```

```
        TextBox4.Text = C.ToString()
```

```
    End If
```

```
End Sub
```

(e) Explain Message Box and Input Box function. Write program for these.

Message Box (MsgBox) and **Input Box (InputBox)** are built-in functions in VB.NET used for interaction between the user and the application. They provide an easy way to display messages and receive input from users.

1. Message Box (MsgBox)

Definition

A **Message Box** is a dialog box used to display messages, warnings, information, or results to the user.

Syntax

```
MsgBox("Message")
```

Example

```
MsgBox("Welcome to VB.NET")
```

Output

```
+-----+
| Welcome to VB.NET |
|      OK          |
+-----+
```

Uses of MsgBox

1. Display information.
2. Show warning messages.
3. Display calculation results.
4. Confirm user actions.

Program Using MsgBox

```
Private Sub Button1_Click(sender As Object,e As EventArgs) Handles Button1.Click
    MsgBox("Hello Student!")
End Sub
```

2. InputBox

Definition

An **InputBox** is a dialog box used to accept input from the user during program execution.

Syntax

```
InputBox("Prompt Message")
```

Example

```
Dim Name As String
Name = InputBox("Enter Your Name")
```

Output

```
+-----+
| Enter Your Name |
|  OK   Cancel   |
+-----+
```

The value entered by the user is returned as a **String**.

Uses of InputBox

1. Accept user input.
2. Receive names, numbers, and text values.
3. Collect data without using TextBox controls.

Program Using InputBox

```
Private Sub Button1_Click(sender As Object, e As EventArgs) Handles Button1.Click

    Dim Name As String

    Name = InputBox("Enter Your Name")

    MsgBox("Welcome " & Name)

End Sub
```

Sample Output

Input :

Rahul

Output :

Welcome Rahul

Program Using Both MsgBox and InputBox

```
Private Sub Button1_Click(sender As Object,
e As EventArgs) Handles Button1.Click

    Dim Age As String

    Age = InputBox("Enter Your Age")

    MsgBox("Your Age is " & Age)
```

End Sub

(f) What is overloading and overriding? Explain it.

1. Method Overloading

Definition

Method Overloading is the process of creating multiple methods with the **same name** but **different parameter lists** (different number, type, or order of parameters) within the same class.

The compiler determines which method to call based on the arguments passed.

Features of Method Overloading

1. Same method name.
2. Different parameter list.
3. Occurs within the same class.
4. Achieves Compile-Time Polymorphism.

Example

Class Addition

```
Public Function Add(ByVal A As Integer,  
                  ByVal B As Integer) As Integer  
    Return A + B  
End Function
```

```
Public Function Add(ByVal A As Integer,  
                  ByVal B As Integer,  
                  ByVal C As Integer) As Integer  
    Return A + B + C  
End Function
```

End Class

Usage

```
Dim Obj As New Addition  
MsgBox(Obj.Add(10, 20))  
MsgBox(Obj.Add(10, 20, 30))
```

Output

30
60

2. Method Overriding

Definition

Method Overriding is the process of redefining a method of the base (parent) class in the derived (child) class using the **Overrides** keyword.

The derived class provides its own implementation of the inherited method.

Features of Method Overriding

1. Requires inheritance.
2. Method name and parameters remain the same.
3. Uses **Overridable** and **Overrides** keywords.
4. Achieves Run-Time Polymorphism.

Example

```
Class Animal

    Public Overridable Sub Sound()
        MsgBox("Animal Makes Sound")
    End Sub

End Class

Class Dog
    Inherits Animal

    Public Overrides Sub Sound()
        MsgBox("Dog Barks")
    End Sub

End Class
```

Usage

```
Dim D As New Dog()
D.Sound()
```

Output

Dog Barks

Part - C

(Long-answer Type Questions)

Answer any four questions of the following:

4. (a) Write a program to connect the customer database using ADO.Net. The program contains ADD, Delete and save Button. Also design the interface. The Customer table contains:

- (i) CustID
- (ii) CustName
- (iii) DESG
- (iv) Basic Salary
- (v) DOB
- (vi) DOJ

Customer Table Structure:

Field Name Data Type

CustID Integer

CustName Varchar

DESG Varchar

BasicSalary Decimal

DOB Date

DOJ Date

CUSTOMER INFORMATION

Customer ID : [TextBox1]

Customer Name : [TextBox2]

Designation : [TextBox3]

Basic Salary : [TextBox4]

Date of Birth : [DateTimePicker1]

Date of Joining : [DateTimePicker2]

[ADD] [DELETE] [SAVE]

DataGridView1

Control	Name
TextBox	TextBox1 (CustID)
TextBox	TextBox2 (CustName)
TextBox	TextBox3 (DESG)
TextBox	TextBox4 (Basic Salary)
DateTimePicker	DateTimePicker1 (DOB)
DateTimePicker	DateTimePicker2 (DOJ)
Button	btnAdd
Button	btnDelete
Button	btnSave
DataGridView	DataGridView1

Imports System.Data.SqlClient

//Database Connection

Dim con As New SqlConnection

con.ConnectionString = _

"Data Source=.;Initial Catalog=CustomerDB;Integrated Security=True"

con.Open()

//ADD Button Code

Private Sub btnAdd_Click(sender As Object,e As EventArgs) Handles btnAdd.Click

Dim cmd As New SqlCommand

cmd.Connection = con

cmd.CommandText = "Insert Into Customer Values('" &
 TextBox1.Text & "','" &
 TextBox2.Text & "','" &
 TextBox3.Text & "','" &
 TextBox4.Text & "','" &
 DateTimePicker1.Value & "','" &
 DateTimePicker2.Value & "')" &

cmd.ExecuteNonQuery()

MsgBox("Record Added Successfully")

End Sub

//DELETE Button Code

Private Sub btnDelete_Click(sender As Object,e As EventArgs) Handles btnDelete.Click

```

Dim cmd As New SqlCommand

cmd.Connection = con

cmd.CommandText = "Delete From Customer Where CustID=" &
    TextBox1.Text

cmd.ExecuteNonQuery()

MsgBox("Record Deleted Successfully")

End Sub

```

//SAVE Button Code

```

Private Sub btnSave_Click(sender As Object,e As EventArgs) Handles btnSave.Click

    Dim da As New SqlDataAdapter
    Dim ds As New DataSet

    da = New SqlDataAdapter("Select * From Customer", con)
    da.Fill(ds, "Customer")
    DataGridView1.DataSource = ds.Tables("Customer")
    MsgBox("Records Saved and Displayed")

End Sub

```

(b) Write a program to move the list of items from one list box to another list box. Also it will remove the selected item from the given any list Box. (Design the Interface)

```

-----

    ListBox1          ListBox2

+-----+          +-----+
| Apple  |          |      |
| Mango  |          |      |
| Orange |          |      |
| Grapes |          |      |
+-----+          +-----+
[ >> ]  [ << ]
[Remove from ListBox1]
[Remove from ListBox2]
-----

```

Move Item from ListBox1 to ListBox2

```

Private Sub btnRight_Click(sender As Object,e As EventArgs) Handles btnRight.Click

    If ListBox1.SelectedIndex <> -1 Then

```

<pre> ListBox2.Items.Add(ListBox1.SelectedItem) ListBox1.Items.Remove(ListBox1.SelectedItem) End If End Sub </pre>
<u>Move Item from ListBox2 to ListBox1</u> <pre> Private Sub btnLeft_Click(sender As Object,e As EventArgs) Handles btnLeft.Click If ListBox2.SelectedIndex <> -1 Then ListBox1.Items.Add(ListBox2.SelectedItem) ListBox2.Items.Remove(ListBox2.SelectedItem) End If End Sub </pre>
<u>Remove Selected Item from ListBox1</u> <pre> Private Sub btnRemove1_Click(sender As Object,e As EventArgs) Handles btnRemove1.Click If ListBox1.SelectedIndex <> -1 Then ListBox1.Items.Remove(ListBox1.SelectedItem) End If End Sub </pre>
<u>Remove Selected Item from ListBox2</u> <pre> Private Sub btnRemove2_Click(sender As Object,e As EventArgs) Handles btnRemove2.Click If ListBox2.SelectedIndex <> -1 Then ListBox2.Items.Remove(ListBox2.SelectedItem) End If End Sub </pre>

(c) What is the difference between error and exception handling in VB.Net. Explain with examples.

During program execution, problems may occur due to incorrect coding or unexpected situations. In VB.NET, these problems are categorized as **Errors** and **Exceptions**. To make applications reliable and user-friendly, VB.NET provides **Exception Handling** mechanisms.

Error

Definition

An **Error** is a mistake in the program that causes incorrect results or prevents the program from compiling and executing properly.

Example

```
Dim Num As Integer
```

```
Num = "ABC"
```

In this example, a string value is assigned to an Integer variable, causing an error.

Characteristics of Errors

1. Caused by programming mistakes.
 2. Usually detected at compile time.
 3. Prevent program execution.
 4. Must be corrected before running the program.
-

Exception

Definition

An **Exception** is an abnormal condition that occurs during program execution (runtime) and interrupts the normal flow of the program.

Example

```
Dim A As Integer = 10
```

```
Dim B As Integer = 0
```

```
Dim C As Integer
```

```
C = A / B
```

Since division by zero is not allowed, a **DivideByZeroException** occurs.

Characteristics of Exceptions

1. Occur during program execution.
2. May terminate the program if not handled.
3. Can be handled using Try-Catch blocks.
4. Improve program reliability when properly managed.

Exception Handling in VB.NET

VB.NET provides the **Try...Catch...Finally** structure to handle exceptions and prevent abrupt program termination.

Syntax

```
Try
    ' Code that may cause exception
Catch ex As Exception
    ' Error handling code
Finally
    ' Executes whether exception occurs or not
End Try
```

Example of Exception Handling

```
Try
    Dim A As Integer = 10
    Dim B As Integer = 0
    Dim C As Integer
    C = A / B
Catch ex As Exception
    MsgBox("Error : " & ex.Message)
Finally
    MsgBox("Program Continues")
End Try
```

Output

Error : Attempted to divide by zero.

Program Continues

Types of Exceptions

1. DivideByZeroException

$C = A / 0$

Occurs when a number is divided by zero.

2. FormatException

Dim X As Integer

X = CInt("ABC")

Occurs when data is converted into an invalid format.

3. IndexOutOfRangeException

Dim Arr(2) As Integer

Arr(5) = 10

Occurs when an invalid array index is used.

4. NullReferenceException

Dim Obj As Object = Nothing

Obj.ToString()

Occurs when an object is used before initialization.

Advantages of Exception Handling

1. Prevents abnormal program termination.
 2. Provides meaningful error messages.
 3. Improves program reliability.
 4. Makes debugging easier.
 5. Enhances user experience.
-

(d) Write short notes on the following:

(i) Timer

Definition

A **Timer** control is used to execute code repeatedly after a specified time interval. It generates a **Tick event** at regular intervals.

Important Properties

- **Interval** – Time interval in milliseconds.
- **Enabled** – Starts or stops the timer.

Example

Timer1.Interval = 1000

Timer1.Enabled = True

Uses

- Digital clocks
- Animations
- Countdown timers
- Automatic updates

(ii) Treeview

A **TreeView** control is used to display hierarchical data in the form of expandable and collapsible nodes.

Important Properties

- **Nodes** – Collection of nodes.
- **SelectedNode** – Currently selected node.

Example

TreeView1.Nodes.Add("India")

TreeView1.Nodes(0).Nodes.Add("Bihar")

Uses

- File Explorer
- Folder structure display
- Organizational charts
- Menu systems

(iii) Event

An **Event** is an action or occurrence recognized by a program, such as a mouse click, key press, or form loading.

Common Events

- Click
- Load
- TextChanged
- MouseHover

Example

```
Private Sub Button1_Click(...) Handles Button1.Click
    MsgBox("Button Clicked")
End Sub
```

Uses

- Responding to user actions
- Event-driven programming
- Interactive applications

(iv) Method

Definition

A **Method** is a block of code that performs a specific task or action on an object.

Common Methods

- Show()
- Hide()
- Clear()
- Focus()

Example

TextBox1.Clear()

Form2.Show()

Uses

- Performing operations on controls
- Executing program logic
- Manipulating objects

(e) What is string in VB Net? Write different types of string classes of VB. Net with examples.

A **String** in VB.NET is a sequence of characters used to store and manipulate text data such as names, addresses, messages, and descriptions. The String data type belongs to the **System.String** class of the .NET Framework.

Example

```
Dim Name As String = "Rahul"
```

In the above example, "Rahul" is a string value.

String Classes in VB.NET

VB.NET mainly provides two classes for working with strings:

1. String Class (System.String)

Definition

The **String** class is used to create and manipulate immutable strings. Immutable means that once a string is created, its value cannot be changed directly.

Common Methods

- Length
- ToUpper()
- ToLower()
- Substring()
- Replace()
- Trim()

Example

```
Dim S As String = "Visual Basic"  
MsgBox(S.ToUpper())
```

Output

VISUAL BASIC

Another Example

```
Dim S As String = "VB.NET"  
MsgBox(S.Length)
```

Output

6

Characteristics

1. Immutable in nature.
2. Easy to use.
3. Suitable for simple string operations.

2. StringBuilder Class (System.Text.StringBuilder)

Definition

The **StringBuilder** class is used to create mutable strings. Mutable means that the string content can be modified without creating a new object.

Common Methods

- Append()
- Insert()
- Remove()
- Replace()

Example

```
Imports System.Text
Dim SB As New StringBuilder("VB")
SB.Append(".NET")
MsgBox(SB.ToString())
```

Output

VB.NET

Another Example

```
Imports System.Text
Dim SB As New StringBuilder("Hello")
SB.Replace("Hello", "Welcome")
MsgBox(SB.ToString())
```

Output

Welcome

Characteristics

1. Mutable in nature.
2. Faster for repeated modifications.
3. Efficient for large strings.

Difference Between String and StringBuilder

String Class	StringBuilder Class
Immutable	Mutable
Creates a new object when modified	Modifies the existing object
Slower for repeated changes	Faster for repeated changes
Suitable for small strings	Suitable for large strings
Namespace: System	Namespace: System.Text

Advantages of String Classes

1. Easy text manipulation.
2. Supports searching and replacing operations.
3. Efficient storage of textual data.
4. Provides numerous built-in methods.

(f) What is COM and Custom Control? Write a program to create a Custom Control.

1. COM (Component Object Model)

Definition

COM (Component Object Model) is a technology developed by Microsoft that enables software components to communicate with each other regardless of the programming language used to create them.

COM components can be reused in different applications and support interoperability between software modules.

Features of COM

1. Reusable software components.
2. Language-independent architecture.
3. Supports component-based development.
4. Easy integration with applications.

Examples

- Microsoft Word Automation
- Microsoft Excel Automation
- ActiveX Controls

2. Custom Control

Definition

A **Custom Control** is a user-defined control created by combining existing controls or by extending the functionality of existing controls.

Custom controls are reusable and can be used in multiple applications.

Advantages of Custom Controls

1. Reusability.
2. Reduces code duplication.
3. Easy maintenance.
4. Improves application development speed.

Step 1: Create a Custom Control Class

```
Public Class MyButton
    Inherits Button

    Public Sub New()

        Me.Text = "Custom Button"
```

```
Me.BackColor = Color.LightBlue  
Me.ForeColor = Color.Black
```

```
End Sub
```

```
End Class
```

Step 2: Use the Custom Control on a Form

```
Public Class Form1
```

```
Private Sub Form1_Load(sender As Object,  
e As EventArgs) Handles MyBase.Load
```

```
Dim Btn As New MyButton
```

```
Btn.Location = New Point(50, 50)
```

```
Me.Controls.Add(Btn)
```

```
End Sub
```

```
End Class
```

Output

```
-----  
| Custom Button |  
-----
```

A customized button appears on the form with predefined properties.

Voc(Sem-IV) —

BCA (CC-9) (2022)

Object Oriented Programming using VB.NET

Time: 3 hours

Full Marks: 50

Part-A

(Multiple-choice Type Questions)

1. Choose the correct answer of the following:

(a) Which Control is used to input the data?

(i) Label

(ii) Text box

(iii) Input box

(iv) Both (ii) and (iii) ✓

(b) The .Net framework is a revolutionary platform that helps you to write the following types of applications:

(i) Windows applications

(ii) Web applications

(iii) Web Services

(iv) All of these ✓

(c) In VB. Net _____ object is immutable.

(i) String ✓

(ii) Text

(iii) Const

(iv) None of these

(d) Fixed values are also called

(i) Global

(ii) Dim

(iii) Literals ✓

(iv) None of these

(e) If we want to execute an application with a specific time interval, we use

(i) Timer Control ✓

(ii) Data and Time Picker

(iii) Text Control

(iv) All of these

2. Fill in the blanks :

- (a) A Destructor is a special member sub of a class that is executed whenever an object of its class goes out of scope.
- (b) Global scope is also known as Public Scope
- (c) An Event is a signal that informs an application that something important has occurred.
- (d) ListBox represents a Windows Control to display a list of items.
- (e) The Load event is used to load a form before it is first displayed.

Part - B

(Short-answer Type Questions)

3. Answer any four questions of the following:

- (a) Write the advantages and disadvantages of using VB. Net Framework.

VB.NET is a modern, object-oriented programming language developed by Microsoft under the .NET Framework. It is widely used for developing Windows, Web, and database applications. The VB.NET Framework provides many facilities that simplify application development.

Advantages of VB.NET Framework

1. Object-Oriented Programming (OOP) Support

VB.NET supports OOP concepts such as inheritance, encapsulation, polymorphism, and abstraction, making programs more organized and reusable.

2. Rapid Application Development (RAD)

VB.NET provides drag-and-drop controls and a visual designer, which helps developers create applications quickly.

3. Rich Framework Class Library (FCL)

It includes a large collection of predefined classes and methods for file handling, database connectivity, networking, and GUI development.

4. Automatic Memory Management

The Common Language Runtime (CLR) automatically manages memory through Garbage Collection, reducing memory leaks.

5. Exception Handling

VB.NET provides structured exception handling using Try-Catch-Finally blocks, improving program reliability.

6. Language Interoperability

Programs written in VB.NET can interact with applications written in other .NET languages such as C#.

7. Security

The .NET Framework provides built-in security features to protect applications and data.

8. Easy Database Connectivity

VB.NET easily connects with databases using ADO.NET for storing and retrieving data.

Disadvantages of VB.NET Framework

1. Slower than Native Code

VB.NET programs run through the CLR, making them slightly slower than native machine-code applications.

2. Requires .NET Framework

Applications cannot run unless the required .NET Framework is installed on the system.

3. Higher Memory Consumption

The CLR and Framework libraries consume additional memory and system resources.

4. Platform Dependency (Traditional .NET Framework)

Older VB.NET applications are mainly designed for Windows platforms.

5. Larger Application Size

Compiled applications may be larger because they depend on .NET libraries.

6. Learning Curve for Beginners

Understanding OOP concepts, CLR, CTS, CLS, and Framework libraries may be difficult for new programmers.

(b) What are the scope of variables in VB. Net? Describe all with examples.

The **scope of a variable** refers to the region of a program where the variable can be accessed or used. In VB.NET, variable scope determines the visibility and lifetime of a variable. Proper use of variable scope helps in managing memory and avoiding programming errors.

Types of Variable Scope in VB.NET

1. Local Scope

A variable declared inside a procedure, function, or event is called a **local variable**. It can be accessed only within that procedure.

Example

```
Private Sub Button1_Click(...) Handles Button1.Click
    Dim Num As Integer = 10
    MsgBox(Num)

End Sub
```

Characteristics

- Accessible only within the procedure.
 - Created when the procedure starts.
 - Destroyed when the procedure ends.
-

2. Module-Level Scope

A variable declared inside a form or module but outside all procedures is called a **module-level variable**.

Example

```
Public Class Form1
    Dim Count As Integer = 0
    Private Sub Button1_Click(...) Handles Button1.Click
        Count = Count + 1
    End Sub
    Private Sub Button2_Click(...) Handles Button2.Click
        MsgBox(Count)
    End Sub
End Class
```

Characteristics

- Accessible by all procedures within the same form or module.
 - Exists as long as the form or module remains active.
-

3. Public (Global) Scope

A variable declared using the **Public** keyword in a module can be accessed from any form or module in the project.

Example

```
Module Module1
    Public CollegeName As String = "ABC College"
End Module
In any form:
MsgBox(CollegeName)
```

Characteristics

- Accessible throughout the application.
 - Can be shared among multiple forms.
 - Useful for storing common data.
-

4. Block Scope

A variable declared inside a block such as **If...End If**, **For...Next**, or **While...End While** is accessible only within that block.

Example

```
If True Then
    Dim X As Integer = 100
    MsgBox(X)
End If
```

Characteristics

- Accessible only inside the block where it is declared.
- Cannot be used outside the block.

Advantages of Variable Scope

1. Improves program organization.
2. Reduces memory usage.
3. Prevents unauthorized access to variables.
4. Makes debugging easier.
5. Enhances program security and reliability.

(c) Write any five string functions which is used in VB. Net. Give examples.

String functions are predefined functions used to perform various operations on strings such as finding length, extracting characters, converting case, and combining strings. These functions make string manipulation easier and more efficient in VB.NET.

1. Len() Function

Purpose

Returns the number of characters in a string.

Syntax

Len(String)

Example

```
Dim Name As String = "VB.NET"
MsgBox(Len(Name))
```

Output

2. UCase() Function

Purpose

Converts all characters of a string into uppercase letters.

Syntax

UCase(String)

Example

Dim Name As String = "vb.net" MsgBox(UCase(Name))
--

Output

VB.NET

3. LCase() Function

Purpose

Converts all characters of a string into lowercase letters.

Syntax

LCase(String)

Example

Dim Name As String = "VB.NET" MsgBox(LCase(Name))
--

Output

vb.net

4. Left() Function

Purpose

Returns a specified number of characters from the left side of a string.

Syntax

Left(String, Number)

Example

```
Dim Name As String = "COMPUTER"
```

```
MsgBox(Left(Name, 4))
```

Output

```
COMP
```

5. Right() Function

Purpose

Returns a specified number of characters from the right side of a string.

Syntax

```
Right(String, Number)
```

Example

```
Dim Name As String = "COMPUTER"
```

```
MsgBox(Right(Name, 4))
```

Output

```
UTER
```

(d) What is validation ? How many types of validations are used in VB. Net?

(d) What is Validation? How Many Types of Validations are Used in VB.NET?

Validation is the process of checking whether the data entered by the user is correct, complete, and in the proper format before it is processed by the application. Validation helps prevent errors and ensures data accuracy and reliability.

Types of Validation in VB.NET

1. Presence Validation

Checks whether a required field has been left blank or not.

Example

```
If TextBox1.Text = "" Then  
    MsgBox("Please enter your name")  
End If
```

Use

Ensures that mandatory fields are filled.

2. Range Validation

Checks whether a value falls within a specified range.

Example

```
Dim Age As Integer
Age = Val(TextBox1.Text)

If Age < 18 Or Age > 60 Then
    MsgBox("Age must be between 18 and 60")
End If
```

Use

Used for marks, age, salary, quantity, etc.

3. Data Type Validation

Checks whether the entered data is of the correct type.

Example

```
If IsNumeric(TextBox1.Text) = False Then
    MsgBox("Please enter numeric value")
End If
```

Use

Ensures that numeric fields contain only numbers.

4. Format Validation

Checks whether data follows a specific format.

Example

```
If Not TextBox1.Text.Contains("@") Then
    MsgBox("Enter a valid Email ID")
End If
```

Use

Used for email addresses, phone numbers, dates, etc.

5. Length Validation

Checks whether the entered data has the required number of characters.

Example

```
If TextBox1.Text.Length <> 10 Then
    MsgBox("Mobile number must contain 10 digits")
End If
```

Use

Used for mobile numbers, PIN codes, passwords, etc.

Advantages of Validation

1. Improves data accuracy.
2. Prevents invalid input.
3. Reduces runtime errors.
4. Enhances application security.
5. Improves user experience.

(e) Create an application which shows the number, which is smallest of three numbers.

Enter First Number : [TextBox1]

Enter Second Number : [TextBox2]

Enter Third Number : [TextBox3]

Smallest Number : [TextBox4]

[Find]

Algorithm

1. Enter three numbers.
2. Store them in variables A, B, and C.
3. Compare the three numbers using If...Else statements.
4. Find the smallest number.
5. Display the result in the output TextBox.

```
Private Sub Button1_Click(sender As Object,  
e As EventArgs) Handles Button1.Click
```

```
    Dim A, B, C As Integer
```

```
    A = Val(TextBox1.Text)
```

```
    B = Val(TextBox2.Text)
```

```
    C = Val(TextBox3.Text)
```

```
    If A <= B And A <= C Then
```

```
        TextBox4.Text = A.ToString()
```

```
    ElseIf B <= A And B <= C Then
```

```
        TextBox4.Text = B.ToString()
```

```
    Else
```

```
        TextBox4.Text = C.ToString()
```

End If
End Sub

(f) What is the difference between checkbox and radio button control ? Create an application for that.

CheckBox	RadioButton
Allows multiple selections.	Allows only one selection from a group.
Independent of other CheckBoxes.	Works as part of a group.
Used when multiple options can be chosen.	Used when only one option should be selected.
Shows a square box.	Shows a circular button.
Example: Hobbies, Skills.	Example: Gender, Payment Mode.

----- Student Information Gender : () Male () Female Hobbies : [] Reading [] Music [] Sports [Show] Result : -----
<pre>Private Sub Button1_Click(sender As Object, e As EventArgs) Handles Button1.Click Dim Gender As String = "" Dim Hobby As String = "" If RadioButton1.Checked Then Gender = "Male"</pre>


```
ElseIf RadioButton2.Checked Then
    Gender = "Female"
End If

If CheckBox1.Checked Then
    Hobby = Hobby & " Reading"
End If

If CheckBox2.Checked Then
    Hobby = Hobby & " Music"
End If

If CheckBox3.Checked Then
    Hobby = Hobby & " Sports"
End If

MsgBox("Gender : " & Gender &
    vbCrLf &
    "Hobbies : " & Hobby)

End Sub
```

Part - C

(Long-answer Type Questions)

4. Answer any four questions of the following:

(a) What are properties, methods and events in VB. Net? Describe it with examples.

In VB.NET, every control and object consists of **Properties, Methods, and Events**. They are the fundamental elements that determine the appearance, behavior, and functionality of an application. By using properties, methods, and events, programmers can create interactive and user-friendly Windows applications.

1. Properties

Properties are the attributes or characteristics of an object that define its appearance and behavior. They can be viewed and modified at design time or run time.

Common Properties

- **Text** – Displays text on a control.
- **Name** – Identifies the control.
- **BackColor** – Sets the background color.
- **ForeColor** – Sets the text color.
- **Visible** – Determines whether the control is visible.

Example

```
TextBox1.Text = "Welcome"  
Button1.BackColor = Color.Yellow
```

In the above example, **Text** and **BackColor** are properties used to change the content and appearance of controls.

Importance of Properties

- Control the look and feel of an application.
- Allow customization of controls.
- Improve user interaction.

2. Methods

Methods are actions or operations that an object can perform. They are used to manipulate controls and execute specific tasks.

Common Methods

- **Show()** – Displays a form.
- **Hide()** – Hides a form.
- **Focus()** – Places the cursor on a control.
- **Clear()** – Removes text from a TextBox.

Example

```
TextBox1.Clear()  
Form2.Show()
```

In the above example, **Clear()** removes the text from the TextBox and **Show()** displays another form.

Importance of Methods

- Perform operations on controls.
- Execute program logic.
- Make applications dynamic and interactive.

3. Events

Events are actions generated by the user or the system. An event occurs when a user interacts with a control or when a specific action takes place.

Common Events

- **Click** – Occurs when a control is clicked.
- **Load** – Occurs when a form loads.
- **MouseHover** – Occurs when the mouse pointer rests on a control.
- **TextChanged** – Occurs when text in a TextBox changes.

Example

```
Private Sub Button1_Click(sender As Object, e As EventArgs) _  
    Handles Button1.Click  
    MsgBox("Button Clicked")  
End Sub
```

When the user clicks the button, the Click event is triggered and a message box is displayed.

Importance of Events

- Handle user interactions.
- Make applications responsive.
- Enable event-driven programming.

Conclusion

Properties, Methods, and Events are the three essential components of VB.NET programming. **Properties** define the characteristics of controls, **Methods** perform operations on controls, and **Events** respond to user or system actions. Together, they form the foundation of event-driven programming and help in developing powerful and interactive Windows applications.

(b) Write and describe any five VB. Net Controls with their properties, methods and events used.

Controls are the graphical elements placed on a form to accept input, display output, and perform various operations. VB.NET provides a rich collection of controls that help in developing interactive and user-friendly applications.

1. Label Control

Purpose

The Label control is used to display text, captions, instructions, or messages to the user. It does not accept user input.

Important Properties

- **Text** – Sets the text displayed on the label.
- **Font** – Changes the font style and size.
- **ForeColor** – Changes the text color.

Method

- **Hide()** – Hides the label from the form.

Event

- **Click** – Occurs when the label is clicked.

Example

```
Label1.Text = "Student Name"  
Label1.Hide()
```

2. TextBox Control

Purpose

The TextBox control is used to accept text input from the user.

Important Properties

- **Text** – Stores or displays the entered text.
- **Multiline** – Allows multiple lines of text.
- **MaxLength** – Limits the number of characters entered.

Method

- **Clear()** – Removes all text from the TextBox.

Event

- **TextChanged** – Occurs when the text changes.

Example

```
TextBox1.Clear()
```

3. Button Control

Purpose

The Button control is used to execute commands when clicked by the user.

Important Properties

- **Text** – Displays the caption on the button.
- **BackColor** – Sets the background color.
- **Enabled** – Enables or disables the button.

Method

- **Focus()** – Sets the focus to the button.

Event

- **Click** – Occurs when the button is clicked.

Example

```
Private Sub Button1_Click(...) Handles Button1.Click  
    MsgBox("Welcome")  
End Sub
```

4. ListBox Control

Purpose

The ListBox control displays a list of items from which the user can select one or more items.

Important Properties

- **Items** – Collection of items in the ListBox.
- **SelectedItem** – Returns the selected item.
- **SelectedIndex** – Returns the index of the selected item.

Methods

- **Items.Add()** – Adds an item.
- **Items.Remove()** – Removes an item.

Event

- **SelectedIndexChanged** – Occurs when the selection changes.

Example

```
ListBox1.Items.Add("Computer")
ListBox1.Items.Remove("Computer")
```

5. ComboBox Control

Purpose

The ComboBox control combines the features of a TextBox and a ListBox. It allows users to select an item from a drop-down list.

Important Properties

- **Text** – Displays the selected item.
- **Items** – Collection of available items.
- **DropDownStyle** – Controls the style of the ComboBox.

Methods

- **Items.Add()** – Adds an item.
- **Items.Clear()** – Removes all items.

Event

- **SelectedIndexChanged** – Occurs when a different item is selected.

Example

```
ComboBox1.Items.Add("VB.NET")
ComboBox1.Items.Add("C#")
```

(c) What is MDI in VB. Net? Why it is used? Also write characteristics and drawbacks for it.

MDI (Multiple Document Interface) is a user interface design in VB.NET that allows multiple child forms (documents) to be opened and managed within a single parent form. The parent form acts as a container, while the child forms are displayed inside it.

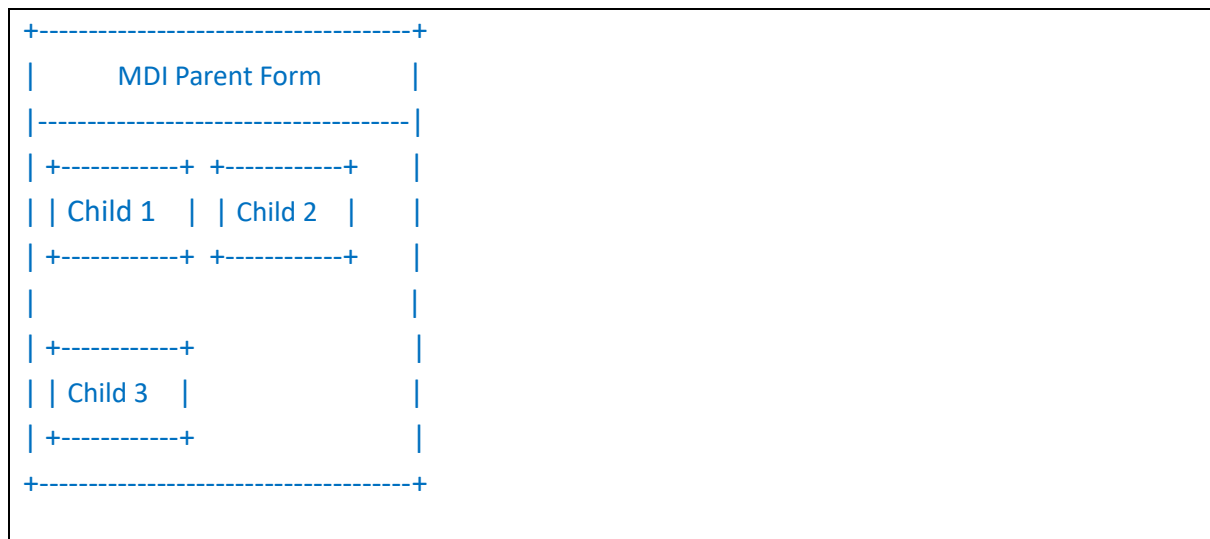
MDI is commonly used in applications where users need to work with multiple documents or windows simultaneously.

Definition

An MDI application consists of:

- **MDI Parent Form** – Main container form.
- **MDI Child Forms** – Forms displayed inside the parent form.

Diagram



Why is MDI Used?

MDI is used when an application needs to manage multiple documents or forms within a single window.

Uses of MDI

1. Allows multiple documents to be opened at the same time.
2. Provides a common menu and toolbar for all child forms.
3. Improves organization of windows.
4. Makes navigation between documents easier.
5. Enhances user productivity.

Examples

- Text Editors
- Database Management Systems

- Visual Studio
- Image Editing Applications

Characteristics of MDI

1. Parent-Child Relationship

An MDI application contains one parent form and multiple child forms.

2. Shared Resources

Menus, toolbars, and status bars can be shared among all child forms.

3. Multiple Windows

Several child forms can remain open simultaneously.

4. Centralized Management

All child forms are controlled through a single parent form.

5. Better Organization

Documents remain within one application window, reducing desktop clutter.

6. Easy Navigation

Users can switch between child forms using menu options such as Cascade and Tile.

Creating an MDI Application in VB.NET

Step 1: Make Parent Form MDI Container

```
Me.IsMdiContainer = True
```

Step 2: Open a Child Form

```
Dim F As New Form2
```

```
F.MdiParent = Me
```

```
F.Show()
```

Advantages of MDI

1. Efficient management of multiple documents.
2. Shared menus and toolbars reduce duplication.
3. Better workspace organization.
4. Convenient for large applications.
5. Improves user productivity.

Drawbacks of MDI

1. Increased Complexity

MDI applications are more difficult to design and maintain than single-form applications.

2. Higher Memory Usage

Multiple child forms consume more system resources.

3. Window Clutter

Opening many child forms may make the workspace crowded.

4. Difficult Navigation

Managing a large number of child windows can become confusing.

5. Reduced Performance

Application performance may decrease when many child forms are open simultaneously.

Difference Between SDI and MDI

SDI	MDI
One document per window	Multiple documents in one parent window
Simple structure	Complex structure
Less memory usage	More memory usage
Easy to develop	More difficult to develop
Example: Notepad	Example: Visual Studio

Conclusion

MDI (Multiple Document Interface) is a technique used in VB.NET to manage multiple child forms inside a single parent form. It provides better organization, shared resources, and improved productivity for applications dealing with multiple documents. However, it increases complexity and memory usage, making it suitable mainly for large and professional applications.

(d) Which components are used in VB. Net Framework? Describe it with example.

The **.NET Framework** is a software development platform developed by **Microsoft** for building and running applications. It provides a rich environment for developing Windows, Web, and Console applications. The major components of the .NET Framework are **CLR, CTS, CLS, and FCL**.

1. CLR (Common Language Runtime)

CLR is the execution engine of the .NET Framework. It manages the execution of programs and provides services such as memory management, security, exception handling, and garbage collection.

Functions of CLR

- Memory Management
- Garbage Collection
- Exception Handling
- Security Management
- Code Execution

Example

```
Dim x As Integer = 10
Dim y As Integer = 20
Dim z As Integer = x + y
```


When this code runs, the CLR manages memory allocation and executes the program.

2. CTS (Common Type System)

CTS defines a common set of data types that all .NET languages must use. It ensures that data types are understood uniformly across different languages.

Examples of CTS Data Types

CTS Type	VB.NET Type
Int32	Integer
String	String
Boolean	Boolean
Double	Double

Example

```
Dim Age As Integer = 20
```

```
Dim Name As String = "Rahul"
```

CTS ensures that these data types are recognized consistently throughout the .NET Framework.

3. CLS (Common Language Specification)

CLS is a set of rules that all .NET languages must follow. It ensures language interoperability, allowing programs written in different .NET languages to work together.

Features

- Language Independence
- Cross-Language Integration
- Standard Programming Rules

Example

A class written in C# can be used in a VB.NET application because both languages follow CLS rules.

4. FCL (Framework Class Library)

FCL is a collection of reusable classes, methods, and components that help developers perform common programming tasks.

Uses of FCL

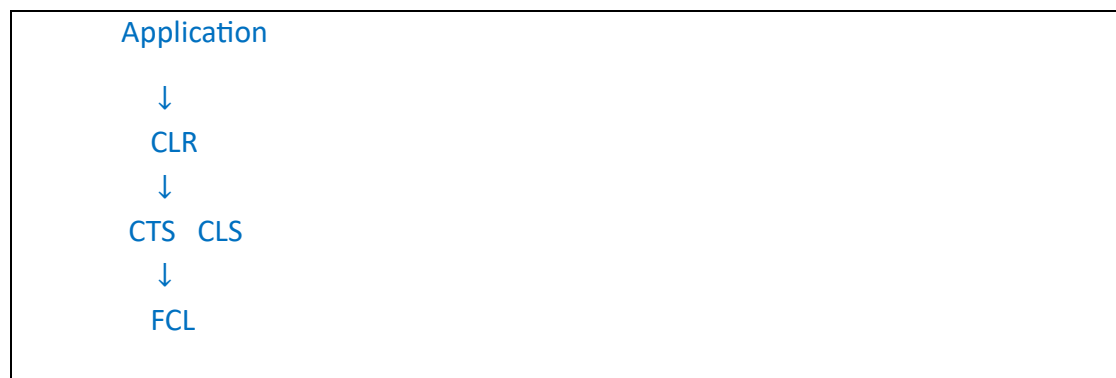
- Input/Output Operations
- File Handling
- Database Connectivity
- String Manipulation
- GUI Development

Example

```
MsgBox("Welcome to VB.NET")
```

The MsgBox() functionality is supported through classes provided by the Framework Class Library.

Architecture of .NET Framework



Advantages of .NET Framework Components

1. Supports multiple programming languages.
2. Provides automatic memory management.
3. Offers a large library of reusable classes.
4. Ensures security and reliability.
5. Facilitates easy application development.

Conclusion

The major components of the VB.NET Framework are **CLR (Common Language Runtime)**, **CTS (Common Type System)**, **CLS (Common Language Specification)**, and **FCL (Framework Class Library)**. These components work together to provide a

secure, efficient, and language-independent environment for developing and executing applications in VB.NET.

(e) What is the difference between Error and Exception? Describe all its types with an example.

While developing a VB.NET application, problems may occur that prevent the program from working correctly. These problems are generally classified as **Errors** and **Exceptions**. Understanding them is important because they affect the execution and reliability of a program.

Error

An **Error** is a mistake made by the programmer that causes the program to produce incorrect results or prevents it from compiling and executing properly.

Errors are generally caused by incorrect syntax, wrong logic, or invalid operations.

Example

```
Dim x As Integer
```

```
x = "ABC"
```

In the above example, a string value is assigned to an Integer variable, resulting in an error.

Exception

An **Exception** is an abnormal condition that occurs during the execution of a program (runtime). Exceptions interrupt the normal flow of the program but can be handled using exception handling techniques.

Example

```
Dim a As Integer = 10
```

```
Dim b As Integer = 0
```

```
Dim c As Integer
```

```
c = a / b
```

Since division by zero is not allowed, a **DivideByZeroException** occurs.

Difference Between Error and Exception

Error	Exception
Caused by programming mistakes.	Caused by unexpected conditions during execution.
Usually detected during compilation.	Usually detected during runtime.
Prevents proper execution of the program.	Interrupts program execution.
Difficult to handle automatically.	Can be handled using Try-Catch blocks.
Example: Syntax mistake.	Example: Division by zero.

Types of Errors in VB.NET

1. Syntax Error

Syntax errors occur when the rules of the programming language are violated.

Example

```
Dim x As Integer
```

Here, the keyword **Integer** is misspelled as **Integar**, causing a compilation error.

Characteristics

- Detected by the compiler.
 - Program cannot execute until corrected.
-

2. Logical Error

Logical errors occur when the program runs successfully but produces incorrect results because of wrong logic.

Example

Area = Length + Width

The correct formula should be:

Area = Length * Width

Characteristics

- Difficult to detect.
 - Program executes normally but gives wrong output.
-

3. Runtime Error

Runtime errors occur while the program is executing.

Example

```
Dim x As Integer = 10
Dim y As Integer = 0

Dim z As Integer = x / y
```

This causes a runtime error because division by zero is not allowed.

Characteristics

- Occurs during execution.
 - May terminate the application if not handled.
-

Common Types of Exceptions in VB.NET

1. DivideByZeroException

Occurs when a number is divided by zero.

```
c = a / 0
```

2. FormatException

Occurs when data is converted into an invalid format.

```
Dim x As Integer
x = CInt("ABC")
```

3. IndexOutOfRangeException

Occurs when an array index exceeds its valid range.

```
Dim arr(2) As Integer
arr(5) = 10
```

4. NullReferenceException

Occurs when an object is used before it is created.

```
Dim obj As Object = Nothing
obj.ToString()
```

Exception Handling in VB.NET

VB.NET provides **Try...Catch...Finally** blocks to handle exceptions and prevent abnormal program termination.

Example

```

Try
    Dim a As Integer = 10
    Dim b As Integer = 0
    Dim c As Integer
    c = a / b
Catch ex As Exception
    MsgBox("Error: " & ex.Message)
Finally
    MsgBox("Program Continues")
End Try

```

Advantages of Exception Handling

1. Prevents sudden program termination.
2. Provides meaningful error messages.
3. Improves program reliability.
4. Makes debugging easier.

Conclusion

Errors and exceptions are common problems encountered during software development. **Errors** are generally caused by mistakes in coding, whereas **exceptions** occur during program execution due to unexpected situations. VB.NET provides powerful exception-handling mechanisms such as **Try-Catch-Finally** to manage exceptions effectively and ensure smooth execution of applications.

(f) Create / develop an application to Input / Output the following:

Input Item code Item name Quantity Price [per unit] Output Item code: Item name:
 Quantity: Price: Total Price: Discount: [10%, if Quantity is more than 100] Net Price:
 (Note: Use the required controls. Discount is calculated by user.)

```

-----
Item Code   : [TextBox1]
Item Name   : [TextBox2]
Quantity    : [TextBox3]
Price/Unit  : [TextBox4]

[Calculate]

Item Code   :
Item Name   :
Quantity    :
Price       :

```

Total Price :
Discount :
Net Price :

Algorithm

1. Accept Item Code, Item Name, Quantity, and Price.
2. Calculate Total Price = Quantity × Price.
3. If Quantity > 100, Discount = 10% of Total Price.
4. Otherwise Discount = 0.
5. Net Price = Total Price – Discount.
6. Display all results.

Private Sub Button1_Click(...) Handles Button1.Click

```
Dim Qty As Integer
Dim Price As Double
Dim Total As Double
Dim Discount As Double
Dim NetPrice As Double

Qty = Val(TextBox3.Text)
Price = Val(TextBox4.Text)

Total = Qty * Price

If Qty > 100 Then
    Discount = Total * 0.1
Else
    Discount = 0
End If

NetPrice = Total - Discount

Label5.Text = TextBox1.Text
Label6.Text = TextBox2.Text
Label7.Text = Qty.ToString()
Label8.Text = Price.ToString()
Label9.Text = Total.ToString()
Label10.Text = Discount.ToString()
Label11.Text = NetPrice.ToString()
```

End Sub

Object Oriented Programming using VB.NET

Time: 3 hours Full Marks: 50

Section - A

1. Choose the correct answer from given options :

(a) The default property for a text box control is:

(i) Text ✓

(ii) Enable

(iii) Multiline

(iv) Password char

(b) The method, which loads the form into memory and displays it on screen:

(i) Load

(ii) Show ✓

(iii) Display

(iv) All of these

(c) Which of the following is the default name assigned to the label1 control in Visual Basic?

(i) Label1 ✓

(ii) LblLabel

(iii) Label0

(iv) New Label

(d) IDE stands for:

(i) Internet Development Environment ✓

(ii) Dual Environment

(iii) Integrated Desktop Environment

(iv) Integrated Development Environment

(e) Which function displays a pop-up window?

(i) MsgBox

(ii) InputBox

(iii) TextBox

(iv) Both (i) and (ii) ✓

2. State True or False:

Statement	True/False
(a) Float data type is not related to VB.NET.	False
(b) When the user clicks a button, an event is triggered.	True
(c) A variable declared inside an event procedure is said to have local scope.	True
(d) The value returned by InputBox is a string.	True
(e) The Solution Explorer does not display form properties.	True

Section - B

3. Answer any four questions of the following:

(a) What is CLR in .NET Framework? Define CTS in VB.NET Framework.

CLR (Common Language Runtime)

CLR is the execution environment of the .NET Framework. It manages the execution of .NET programs and provides various services to applications.

Functions of CLR

1. Memory Management
2. Exception Handling
3. Garbage Collection
4. Security Management
5. Code Execution

Example: When a VB.NET program runs, CLR converts Intermediate Language (IL) code into machine code and executes it.

CTS (Common Type System)

CTS is a standard that defines how data types are declared, used, and managed in the .NET Framework. It ensures that objects written in different .NET languages can interact with each other.

Examples of CTS Data Types

CTS Type VB.NET Type

Int32 Integer

String String

Boolean Boolean

Double Double

Conclusion

CLR executes .NET applications, while CTS provides a common set of data types for all .NET languages.

(b) Design GUI and Write Code to Add and Remove Items from a ListBox

GUI Design

```
-----  
Enter Item : [TextBox1]  
[ListBox1]  
[Add Button] [Remove Button]  
-----
```

VB.NET Code

Add Item

```
Private Sub Button1_Click(...) Handles Button1.Click  
    ListBox1.Items.Add(TextBox1.Text)  
End Sub
```

Remove Selected Item

```
Private Sub Button2_Click(...) Handles Button2.Click  
    If ListBox1.SelectedIndex <> -1 Then  
        ListBox1.Items.Remove(ListBox1.SelectedItem)  
    End If  
End Sub
```

Output

- Add button inserts items into ListBox.
- Remove button deletes the selected item.

(c) Explain Any Four Data Types in VB.NET with Examples

Data types specify the kind of data that a variable can store.

Data Type	Description	Example
Integer	Stores whole numbers	Dim Age As Integer = 20
Double	Stores decimal values	Dim Salary As Double = 25000.50
String	Stores text data	Dim Name As String = "Rahul"
Boolean	Stores True or False	Dim Result As Boolean = True

Example Program

```
Dim Age As Integer = 18
```

```
Dim Marks As Double = 85.5
```

```
Dim Name As String = "Amit"
```

```
Dim Pass As Boolean = True
```

(d) Explain Any Four Features of .NET Framework

1. Language Independence

Supports multiple languages such as VB.NET, C#, and F#.

2. Platform Interoperability

Applications written in different .NET languages can communicate with each other.

3. Automatic Memory Management

CLR automatically manages memory using Garbage Collection.

4. Security

Provides built-in security features such as code access security and authentication.

Conclusion

The .NET Framework provides a secure, reliable, and language-independent environment for application development.

(e) Design GUI and Write Code to Change Background Colour and Text Colour of a Button

GUI Design

```
-----  
[Button1]  
[Background Colour]  
[Text Colour]  
-----
```

VB.NET Code

Change Background Colour

```
Private Sub Button2_Click(...) Handles Button2.Click  
    Button1.BackColor = Color.Yellow  
End Sub
```

Change Text Colour

```
Private Sub Button3_Click(...) Handles Button3.Click  
    Button1.ForeColor = Color.Red  
End Sub
```

Output

- Clicking "Background Colour" changes button background colour.
- Clicking "Text Colour" changes button text colour.

(f) Explain Any Two Events of Button Control with Examples

1. Click Event

The Click event occurs when the user clicks a button.

Example

```
Private Sub Button1_Click(...) Handles Button1.Click  
    MsgBox("Button Clicked")  
End Sub
```

Output: Displays a message when the button is clicked.

2. MouseHover Event

The MouseHover event occurs when the mouse pointer is placed over the button.

Example

```
Private Sub Button1_MouseHover(...) Handles Button1.MouseHover
    MsgBox("Mouse is over Button")
End Sub
```

Output: Displays a message when the mouse pointer hovers over the button.

Section - C

4. Answer any four questions of the following:

(a) Explain different types of inheritance in VB.NET with examples.

Inheritance is a core pillar of Object-Oriented Programming (OOP) that allows a child class (derived class) to inherit the properties, methods, and fields of an existing parent class (base class), promoting code reusability. In VB.NET, the Inherits keyword is used.

1. Single Inheritance

A derived class inherits from exactly **one** single base class. This is the most common form of inheritance.

```
VB.Net
' Base Class
Public Class Animal
    Public Sub Eat()
        Console.WriteLine("Eating...")
    End Sub
End Class

' Derived Class
Public Class Dog : Inherits Animal
    Public Sub Bark()
        Console.WriteLine("Barking...")
    End Sub
End Class
```

2. Hierarchical Inheritance

Multiple derived classes inherit from a **single** common base class.

```
VB.Net

Public Class Shape
    Public Width As Integer
End Class

Public Class Rectangle : Inherits Shape
    ' Inherits Width
End Class

Public Class Circle : Inherits Shape
    ' Inherits Width
End Class
```

3. Multilevel Inheritance

A derived class inherits from a base class, which itself is a derived class of another base class (forming a chain of inheritance).

```
VB.Net

Public Class Person
    Public Sub Walk() : End Sub
End Class

Public Class Employee : Inherits Person
    Public Sub Work() : End Sub
End Class

Public Class Manager : Inherits Employee
    Public Sub Direct() : End Sub
End Class
```

Note on Multiple Inheritance: VB.NET **does not support** Multiple Inheritance (inheriting from more than one class directly) due to structural ambiguity ("Diamond Problem"). Instead, multiple inheritance is achieved using **Interfaces**.

(b) Describe properties and events of Timer control with examples.

The **Timer Control** is a background component used to execute code at specified time intervals continuously without blocking the user interface thread.

1. Key Properties

- **Interval:** Specifies the time duration in **milliseconds** (\$1000 \text{ ms} = 1 \text{ second}\$) that must elapse before the timer executes.
- **Enabled:** A boolean flag (True/False) that toggles the timer on or off.

2. Default Event

- **Tick:** This event is raised automatically every time the time period specified in the Interval property elapses, provided Enabled is set to True.

Code Example: A Digital Clock Form

```
Public Class FormClock
    ' Form initialization
    Private Sub FormClock_Load(sender As Object, e As EventArgs) Handles MyBase.Load
        ' Configure properties programmatically
        Timer1.Interval = 1000 ' 1 second
        Timer1.Enabled = True ' Start Timer
    End Sub

    ' Event handler called every second
    Private Sub Timer1_Tick(sender As Object, e As EventArgs) Handles Timer1.Tick
        ' Update UI text
        lblTime.Text = DateTime.Now.ToString("HH:mm:ss")
    End Sub
End Class
```

(c) Design GUI and write VB.Net code to accept marks of five subjects and calculate total and percentage.

GUI:

MARKSHEET	
Subject1 :	[TextBox1]
Subject2 :	[TextBox2]
Subject3 :	[TextBox3]
Subject4 :	[TextBox4]
Subject5 :	[TextBox5]
Total :	[TextBox6]

Percentage: [TextBox7]

[Calculate Button]

CLI:

```
Private Sub Button1_Click(sender As Object,  
e As EventArgs) Handles Button1.Click
```

```
    Dim m1, m2, m3, m4, m5 As Integer
```

```
    Dim total As Integer
```

```
    Dim per As Double
```

```
    m1 = Val(TextBox1.Text)
```

```
    m2 = Val(TextBox2.Text)
```

```
    m3 = Val(TextBox3.Text)
```

```
    m4 = Val(TextBox4.Text)
```

```
    m5 = Val(TextBox5.Text)
```

```
    total = m1 + m2 + m3 + m4 + m5
```

```
    per = total / 5
```

```
    TextBox6.Text = total.ToString()
```

```
    TextBox7.Text = per.ToString()
```

```
End Sub
```

(d) Explain interface in VB.NET.

An Interface is a collection of method declarations, properties, and events without implementation. It provides a contract that implementing classes must follow.

Syntax

```
Interface IStudent
```

```
    Sub Study()
```

```
End Interface
```

Implementing Interface

```
Interface IStudent
```

```
    Sub Study()
```

```
End Interface
```



```
Class Student
    Implements IStudent

    Public Sub Study() Implements IStudent.Study
        MsgBox("Studying")
    End Sub
End Class
```

Features of Interface

1. Contains only declarations.
2. Supports multiple inheritance.
3. Improves code flexibility.
4. Promotes abstraction.

Advantages

- Better code organization.
- Loose coupling.
- Reusability.
- Easy maintenance.

Conclusion

Interfaces provide a standard structure that implementing classes must follow and are widely used in OOP development.

(e) Distinguish between Single Document interface and Multiple document interface.

Basis	SDI (Single Document Interface)	MDI (Multiple Document Interface)
Definition	Supports one document per window.	Supports multiple documents within one parent window.
Window Structure	Each document has its own separate window.	Multiple child windows exist inside a single parent window.
User Interface	Simple and easy to use.	More complex but powerful.

Basis	SDI (Single Document Interface)	MDI (Multiple Document Interface)
Memory Usage	Requires less memory.	Requires comparatively more memory.
Development	Easier to design and implement.	More difficult to design and manage.
Document Management	Documents are managed independently.	Documents are managed under a common parent window.
Switching Between Documents	User switches between separate windows.	User switches between child windows inside the same application.
Menu and Toolbar	Each window has its own menu and toolbar.	Parent window can share menus and toolbars among child windows.
Suitable For	Small and simple applications.	Large applications handling multiple documents.
Examples	Notepad, Paint, Calculator.	Visual Studio, Adobe Photoshop (older versions), Microsoft Excel (older versions).

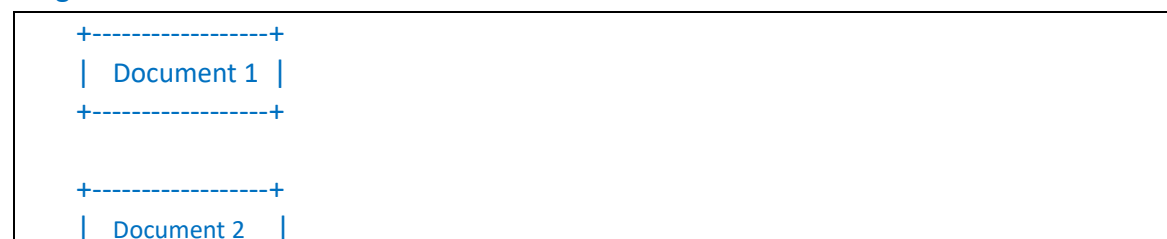
Advantages of SDI

1. Simple and user-friendly.
2. Easy to develop and maintain.
3. Lower memory consumption.

Advantages of MDI

1. Better organization of multiple documents.
2. Easy sharing of menus and resources.
3. Improved productivity when working with several files.

Diagram of SDI



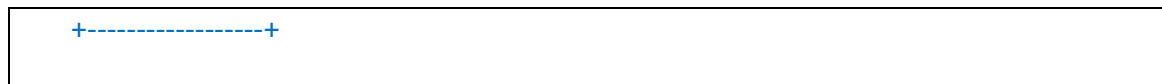
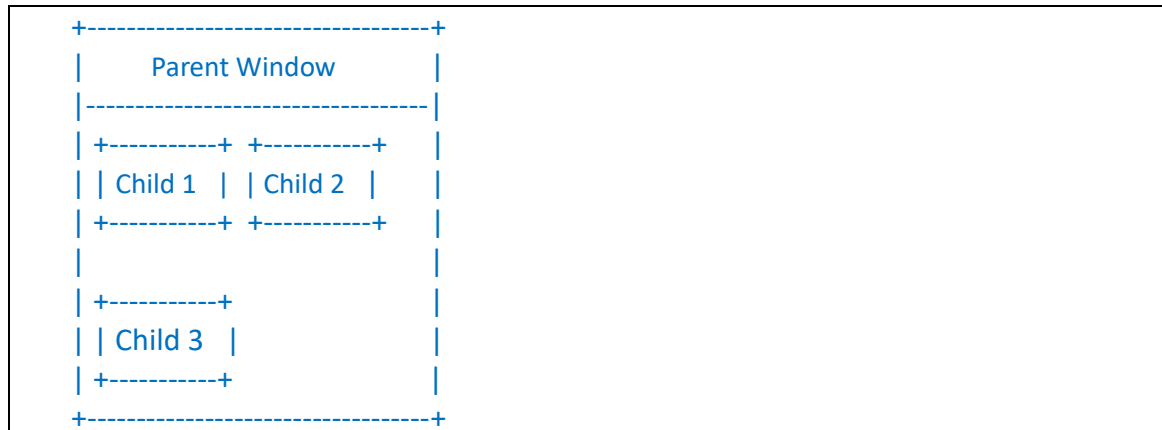


Diagram of MDI



(f) Explain procedures, functions and events in VB.NET.

1. Procedures (Sub Procedures)

A **Procedure** (declared with Sub) is a block of code enclosed between Sub and End Sub that performs a series of specific commands but **does not return a value** to the calling code block.

```

Public Sub DisplayWelcomeMessage(ByVal userName As String)

    Console.WriteLine("Welcome back, " & userName)

End Sub ' Logic finishes, no value sent back

```

2. Functions

A **Function** is a block of code enclosed between Function and End Function that processes a routine and explicitly **returns a single result value** to the calling routine using a Return statement.

```

Public Function AddNumbers(ByVal val1 As Integer, ByVal val2 As Integer) As Integer

    Dim sum As Integer = val1 + val2

    Return sum ' Sends the computed calculation back

End Function

```

3. Events

An **Event** is an asynchronous signal raised by an object (like a button click or timer interval completion) to alert the system that an action has taken place. Code blocks called **Event Handlers** hook into these signals using the Handles keyword to respond to user interactions programmatically.

```
' The Handles clause binds this procedure directly to the button's Click event signal

Private Sub btnSubmit_Click(sender As Object, e As EventArgs) Handles
    btnSubmit.Click

    MessageBox.Show("Submit event detected and processed successfully!")

End Sub
```

Voc (Sem-IV) - BCA (CC-9) (2024)

Object Oriented Programming using VB.NET

Time: 3 hours Full Marks : 70

Section - A

1. Choose the correct answer of the following:

(a) Which of the following displays the list of projects contained in the Visual Basic current solution?

- (i) List Window
- (ii) Project Window
- (iii) Catalogue Window
- (iv) Solution Explorer Window ✓

(b) Which of the following instruction tells the computer to close the current form in Visual Basic?

- (i) This. Close()
- (ii) Me. Close() ✓
- (iii) Close. this()
- (iv) Close()

(c) Object respond to an event by writing

(i) Defining events

(ii) Operations

(iii) Defining actions

(iv) Event procedures ✓

(d) Which of the following is the default name assigned to the label control in Visual Basic?

(i) Label 1 ✓

(ii) Default Label

(iii) Label()

(iv) New Label

(e) In Visual Basic, which of the following keyword tells the computer to pass the variable's address rather than its contents?

(i) ByVal

(ii) ByVal

(iii) ByRef ✓

(iv) ByVal

(f) Which of the following returns a value after performing its specific task in Visual Basic?

(i) Structure

(ii) Sub block

(iii) Sub procedure

(iv) Function procedure ✓

(g) Which task is accomplished in the code editor?

(i) Adding forms to the project

(ii) Adding controls to the form

(iii) Adding event procedures to the form ✓

(iv) Adding controls to the form

(h) Which is not a common control event?

(i) Click

(ii) Single click ✓

(iii) Double click

(iv) MouseMove

(i) Which function displays a pop-up window?

(i) MsgBox

(ii) TextBox

(iii) InputBox

(iv) Both (i) and (iii) ✓

(j) Which value for the ComboBox DropDown Style property allows a user to type in data?

(i) DropDown ✓

(ii) DropDown Simple

(iii) DropDown List

(iv) None of these

2. State True or False of the following:

Question	Answer
(a) Select Case is an example of multiple branches from single expression.	True
(b) Exit Form is not a valid statement.	True
(c) CheckedChanged event is activated when a RadioButton is selected.	True
(d) When a parameter in a procedure is declared ByVal, a copy of the argument is sent to the procedure.	True
(e) The Visual Basic Code Editor will automatically detect certain types of errors as you are entering code.	True
(f) A variable declared outside of an event procedure is said to have class level scope.	True
(g) Properties can be viewed in two ways – Alphabetic and Categorized.	True
(h) By default, the TextBox control can hold text as a single line.	True

Section B

3. Answer any four questions of the following:

(a) What do you understand by namespace? Give example.

A **Namespace** is a collection of related classes, interfaces, structures, and modules. It helps organize code and prevents naming conflicts between classes having the same name.

Example:

```
Namespace MyApplication
Class Student
    Public Name As String
End Class
End Namespace
```

Here, MyApplication is a namespace and Student is a class inside it.

Advantages:

- Organizes code properly.
- Avoids name conflicts.
- Makes applications easier to manage.

(b) What is CLR? (c) What is purpose of MustOverride keyword ?

CLR (Common Language Runtime) is the execution environment of the .NET Framework. It manages the execution of .NET programs.

Functions of CLR:

1. Memory management.
2. Garbage collection.
3. Exception handling.
4. Security management.
5. Code execution and verification.

Example:

When a VB.NET program is executed, the CLR converts the Intermediate Language (IL) code into machine code and runs it.

(c) What is the purpose of MustOverride keyword?

The **MustOverride** keyword is used in an abstract base class to declare a method that must be implemented by derived classes.

- It does not contain any code.
- Derived classes are required to override it.

Example:

```
MustInherit Class Shape
    Public MustOverride Sub Draw()
End Class
Class Circle
    Inherits Shape

    Public Overrides Sub Draw()
```

```
        MsgBox("Drawing Circle")
    End Sub
End Class
```

Here, the Draw() method must be implemented in the Circle class.

(d) What are the different types of errors ? Describe On Error Go To statement with an example.

Syntax Errors: These occur when the code violates the grammatical rules of the VB.NET language (e.g., misspelled keywords, missing brackets). They are caught immediately by the IDE compiler before the code runs.

Runtime Errors: These occur while the application is executing. The syntax is perfect, but an illegal operation is requested (e.g., dividing a number by zero, attempting to open a non-existent file). These cause unexpected crashes if unhandled.

Logical Errors (Bugs): The program runs without crashing, but outputs incorrect results because the underlying algorithm or mathematical logic is flawed (e.g., using + instead of *).

On Error GoTo is a legacy, unstructured error-handling mechanism carried over from traditional Visual Basic. It instructs the program to jump to a specific labeled line of code if a runtime error occurs, preventing an immediate crash.

Syntax:

On Error GoTo ErrorHandler

```
Private Sub Button1_Click()
    On Error GoTo ErrorHandler

    Dim x As Integer = 10
    Dim y As Integer = 0
    Dim z As Integer

    z = x / y

    Exit Sub

ErrorHandler:
    MsgBox("Error: Division by Zero")
End Sub
```

(e) What is Class library? Write steps to create the class library.

A **Class Library** is a collection of reusable classes, methods, and components that can be used in multiple applications.

Steps to Create a Class Library:

1. Open **Visual Studio**.
2. Click **File → New → Project**.
3. Select **Class Library** project type.
4. Enter project name and click **Create/OK**.
5. Write classes and methods in the library.
6. Build the project using **Build → Build Solution**.
7. A .dll file is generated.
8. Add the DLL reference to other applications to use the library.

Benefits:

- Code reusability.
- Easy maintenance.
- Reduces development time.

(f) What are the various methods, properties and events of Timer control?

1. Key Properties

- **Interval:** Specifies the time period (expressed in **milliseconds**) that must elapse before the timer raises its action event. For example, setting it to 1000 equals exactly 1 second.
- **Enabled:** A boolean value (True/False) that turns the timer on or off. If set to True, the timer starts counting; if False, it goes dormant.

Example:

```
Timer1.Interval = 1000  
Timer1.Enabled = True
```

2. Key Methods

- **Start():** Activates the timer component by programmatically setting the Enabled property to True.
- **Stop():** Deactivates the timer component by programmatically setting the Enabled property to False.

Example:

```
Timer1.Start()  
Timer1.Stop()
```

3. Key Events

- **Tick:** This is the default and most important event of the Timer control. It fires automatically every time the defined Interval period elapses, provided the component remains enabled.

Example:

```
Private Sub Timer1_Tick(sender As Object, e As EventArgs) Handles Timer1.Tick
```

```
Label1.Text = TimeOfDay  
End Sub
```

Section - C

4. Answer any four questions of the following:

(a) Describe shared methods and shared variables with examples.

In VB.NET, the Shared keyword is used to declare class-level members. By default, class members are instance-level (each object gets its own copy). Marking a member as Shared changes its scope so that it belongs to the class itself rather than to instances of the class.

1. Shared Variables

A **Shared Variable** (also known as a class variable) is allocated memory only once. No matter how many objects are instantiated from that class, they all share this single copy of the variable. If one instance modifies a shared variable, the updated value is immediately visible to all other instances.

2. Shared Methods

A **Shared Method** (also known as a class method) can be invoked directly using the class name without requiring an object instance to be created first.

- **Critical Rule:** Shared methods can only access other Shared variables or invoke other Shared methods directly. They cannot use the Me keyword or access instance-level variables unless they explicitly create or receive an object instance.

```
Public Class Student  
    ' Shared Variable: Tracks total student enrollment across the system  
    Public Shared TotalStudents As Integer = 0  
  
    ' Instance Variables: Unique to each individual student object  
    Public StudentName As String  
    Public RollNo As Integer  
  
    ' Constructor  
    Public Sub New(ByVal name As String, ByVal roll As Integer)  
        Me.StudentName = name  
        Me.RollNo = roll  
        ' Increment the shared counter every time a new student is registered  
        TotalStudents += 1  
    End Sub
```

```

' Shared Method: Can be executed without creating an object
Public Shared Sub DisplayEnrollmentCount()
    Console.WriteLine("Total Students Enrolled: " & TotalStudents)
End Sub
End Class

' Execution Module
Module Module1
    Sub Main()
        ' Accessing shared method before any object is created
        Student.DisplayEnrollmentCount() ' Outputs: 0

        ' Instantiating two separate student objects
        Dim s1 As New Student("Amit", 101)
        Dim s2 As New Student("Priya", 102)

        ' Accessing the shared method using the Class Name
        Student.DisplayEnrollmentCount() ' Outputs: 2
    End Sub
End Module

```

(b) What is an Abstract class ? Differentiate between Abstract class and Interface in Vb. Net.

Abstract Class

An **Abstract Class** in VB.NET is declared using the `MustInherit` modifier. It serves as a partial blueprint meant exclusively for inheritance; it **cannot be instantiated** directly using the `New` keyword. It can contain a mix of unimplemented methods (`MustOverride`) and fully implemented methods.

Interface

An **Interface** is declared using the `Interface` keyword. It represents a strict contract that implementing classes must adhere to. It contains **no implementation logic, no variables, and no constructors**. It only defines signatures for methods, properties, events, or indexers.

Key Structural Differences

Feature	Abstract Class (<code>MustInherit</code>)	Interface (<code>Interface</code>)
Implementation	Can contain both complete methods	Cannot contain any implementation code.

	(with code) and abstract methods.	All members are purely structural signatures.
Inheritance Model	A class can inherit from only one base class (Single Inheritance).	A class can implement multiple interfaces simultaneously.
Fields & Variables	Can define member variables (fields), constants, and fields of varying access levels.	Cannot contain fields, variables, or constants.
Access Modifiers	Members can use any access modifier (Public, Protected, Private, Friend).	All members are implicitly Public. You cannot explicitly apply access modifiers inside it.
Constructors	Can contain constructors (Sub New) to initialize base-class state.	Cannot contain constructors.

(c) Design an interface and write VB.NET code to enter ten numbers in an array and display their sum.

UI Layout Blueprint

TextBox (txtNumber): Accepts individual integer inputs.

Button (btnAdd): Appends the input number to the fixed-size array.

Button (btnCalculate): Processes the loop to sum the array values.

Label (lblStatus): Displays real-time array tracking (e.g., "3 / 10 elements entered").

Label (lblResult): Outputs the final mathematical sum.

```
Public Class FormArraySum
    ' Global Declarations: Fixed-size array for 10 integers (0 to 9)
    Private numbers(9) As Integer
    Private currentIndex As Integer = 0

    Private Sub btnAdd_Click(sender As Object, e As EventArgs) Handles btnAdd.Click
        ' Bound Check: Ensure we don't overflow the array bounds
        If currentIndex < 10 Then
            Dim inputVal As Integer

            ' Input Validation
```

```

    If Integer.TryParse(txtNumber.Text, inputVal) Then
        numbers(currentIndex) = inputVal
        currentIndex += 1

        ' Update UI Feedback
        lblStatus.Text = $"Elements entered: {currentIndex} of 10"
        txtNumber.Clear()
        txtNumber.Focus()
    Else
        MessageBox.Show("Please input a valid whole number.", "Validation Error",
        MessageBoxButtons.OK, MessageBoxIcon.Error)
        Exit Sub
    End If
End If

' If array is fully populated, toggle control availability
If currentIndex = 10 Then
    txtNumber.Enabled = False
    btnAdd.Enabled = False
    lblStatus.Text = "Array Full. Ready to calculate sum."
    btnCalculate.Focus()
End If
End Sub

Private Sub btnCalculate_Click(sender As Object, e As EventArgs) Handles btnCalculate.Click
    ' Validation: Ensure array is filled before summing
    If currentIndex < 10 Then
        MessageBox.Show("Please finish entering all 10 numbers first.", "Incomplete Data",
        MessageBoxButtons.OK, MessageBoxIcon.Warning)
        Exit Sub
    End If

    Dim totalSum As Integer = 0

    ' Loop through the filled array to accumulate sum
    For i As Integer = 0 To numbers.Length - 1
        totalSum += numbers(i)
    Next

    ' Display Output
    lblResult.Text = "Calculated Sum: " & totalSum.ToString()
End Sub
End Class

```

(d) What is List View Control ? Design proper interface and write VB. NET-code to insert the records containing student's Roll, Name, Email and COURSE in the ListView control.

The ListView control is a highly versatile Windows Forms control used to display collections of data items. It supports multiple layout styles (Large Icons, Small Icons, List, Details). To build data grids, it is set to **Details View**, which organizes fields cleanly into rows and columns complete with headers.

UI Layout Blueprint

To handle student records, design a form incorporating these control assets:

1. **ListView (lvStudents):** Main display area. Properties to change in the Properties window:
 - View = Details
 - GridLines = True
 - FullRowSelect = True
2. **Four TextBoxes:** txtRoll, txtName, txtEmail, txtCourse.
3. **One Button:** btnInsertRecord.

```
Public Class FormStudentRegistry

    Private Sub FormStudentRegistry_Load(sender As Object, e As EventArgs)
        Handles MyBase.Load
            ' Configure columns programmatically to ensure precision during runtime
            execution
            lvStudents.View = View.Details
            lvStudents.GridLines = True
            lvStudents.FullRowSelect = True

            ' Adding Column Headers with explicit width allocation
            lvStudents.Columns.Add("Roll No", 80, HorizontalAlignment.Left)
            lvStudents.Columns.Add("Student Name", 150, HorizontalAlignment.Left)
            lvStudents.Columns.Add("Email Address", 180, HorizontalAlignment.Left)
            lvStudents.Columns.Add("Enrolled Course", 120, HorizontalAlignment.Left)
        End Sub

    Private Sub btnInsertRecord_Click(sender As Object, e As EventArgs) Handles
        btnInsertRecord.Click
            ' 1. Form Validation validation
            If String.IsNullOrEmpty(txtRoll.Text) OrElse
                String.IsNullOrEmpty(txtName.Text) OrElse
                String.IsNullOrEmpty(txtEmail.Text) OrElse
                String.IsNullOrEmpty(txtCourse.Text) Then

                MessageBox.Show("All input fields are mandatory.", "Missing Fields",
                    MessageBoxButtons.OK, MessageBoxIcon.Warning)
```

```

        Exit Sub
    End If

    ' 2. Create the primary ListViewItem row using the first column entry (Roll
    No)
    Dim mainItem As New ListViewItem(txtRoll.Text.Trim())

    ' 3. Append subsequent column values as SubItems to the parent item
    mainItem.SubItems.Add(txtName.Text.Trim())
    mainItem.SubItems.Add(txtEmail.Text.Trim())
    mainItem.SubItems.Add(txtCourse.Text.Trim())

    ' 4. Commit the item row to the ListView item collection
    lvStudents.Items.Add(mainItem)

    ' 5. UI Cleanup for next input operation
    ClearInputFields()
End Sub

Private Sub ClearInputFields()
    txtRoll.Clear()
    txtName.Clear()
    txtEmail.Clear()
    txtCourse.Clear()
    txtRoll.Focus()
End Sub
End Class

```

(e) What is user control? Write steps to create a user control in VB. NET.

What is a User Control?

A **User Control** (UserControl) is a custom composite control designed by combining existing standard Windows Forms controls (like buttons, text boxes, and labels) into a single reusable object. It inherits from System.Windows.Forms.UserControl. It enables developers to modularize complex interface segments and re-use them effortlessly across multiple forms or separate projects without rewriting underlying code.

Step-by-Step Guide to Creating and Using a User Control

1. **Create/Open Project:** Open your Windows Forms Application project within Microsoft Visual Studio.

2. **Add User Control Asset:** Right-click your project name in the **Solution Explorer** panel → Select **Add** → Click on **User Control (Windows Forms)**. Name the control file (e.g., ExtendedLoginControl.vb) and click **Add**.
3. **Design Layout:** A blank design container will open. From the Toolbox panel, drag and drop existing constituent components onto this designer grid (e.g., drag two Labels, two TextBoxes for Username/Password, and a Login Button). Arrange them visually.
4. **Implement Code Functionality:** Double-click the embedded button to access the code-behind file. Add custom events, properties, or verification functions inside the control class.
5. **Compile the Assembly:** Select **Build** from the main application menu bar and click **Build Solution** (or press Ctrl+Shift+B). Compiling registers the new custom control into your development environment.
6. **Consume the Control:** Open your standard execution target form (e.g., Form1.vb). Open your **Toolbox** panel. At the top, a new custom group bearing your project name will be present. Drag your newly built ExtendedLoginControl directly onto your standard Form layout just like any native control.

(f) What is ADO.NET ? Describe its architecture.

ADO.NET (ActiveX Data Objects .NET) is a data access technology of the .NET Framework used to connect applications with databases. It provides classes and methods for retrieving, updating, inserting, and deleting data from various data sources such as SQL Server, Oracle, MS Access, and XML files.

ADO.NET is designed to work efficiently in both **Connected** and **Disconnected** environments.

ADO.NET is a collection of classes in the .NET Framework that enables communication between a VB.NET application and a database.

Namespace

Imports System.Data

Imports System.Data.SqlClient

Features of ADO.NET

1. Supports Connected and Disconnected Architecture.
2. Fast data access.
3. Supports XML integration.
4. Platform independent within .NET.
5. Secure and scalable database connectivity.

Architecture of ADO.NET

ADO.NET architecture consists of two main components:

1. .NET Data Provider (Connected Architecture)

Used for direct communication with the database.

Components:

(a) Connection

Establishes connection with the database.

```
Dim con As New SqlConnection()
```

(b) Command

Executes SQL queries and stored procedures.

```
Dim cmd As New SqlCommand()
```

(c) DataReader

Reads data in a forward-only, read-only manner.

```
Dim dr As SqlDataReader
```

(d) DataAdapter

Acts as a bridge between the DataSet and the database.

```
Dim da As New SqlDataAdapter()
```

2. DataSet (Disconnected Architecture)

A DataSet stores data temporarily in memory and allows operations without maintaining a continuous database connection.

Components:

(a) DataSet

Stores one or more tables in memory.

```
Dim ds As New DataSet()
```

(b) DataTable

Represents a table inside the DataSet.

(c) DataRow

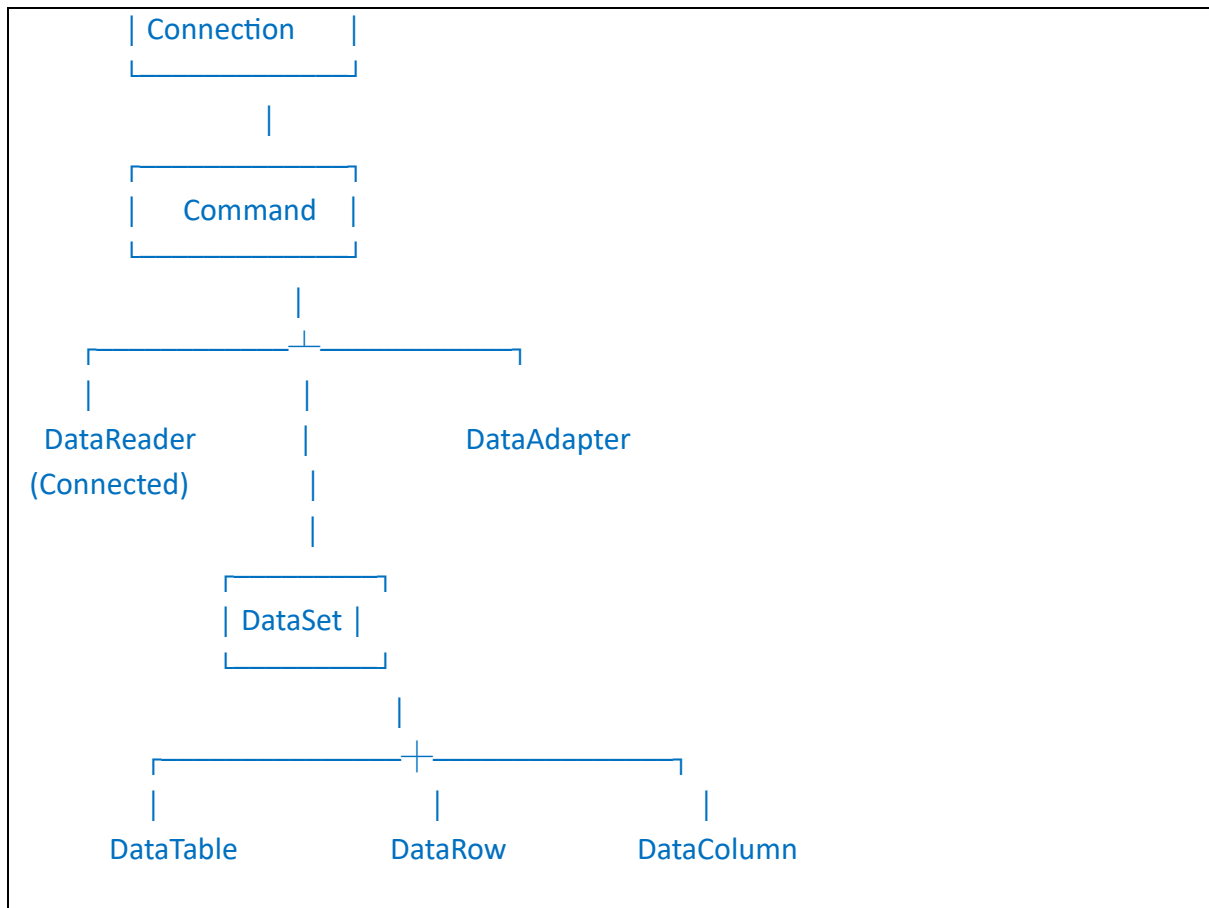
Represents a row in a DataTable.

(d) DataColumn

Represents a column in a DataTable.

ADO.NET Architecture Diagram





Connected Architecture

- Uses Connection, Command, and DataReader.
- Database connection remains open while accessing data.
- Faster for real-time operations.

Example

```
Dim con As New SqlConnection(constr)
con.Open()
Dim cmd As New SqlCommand("Select * From Customer", con)
Dim dr As SqlDataReader
dr = cmd.ExecuteReader()
```

Disconnected Architecture

- Uses DataAdapter and DataSet.
- Connection is opened only when required.

- Data is stored in memory and processed offline.

Example

```
Dim da As New SqlDataAdapter("Select * From Customer", con)
```

```
Dim ds As New DataSet  
da.Fill(ds, "Customer")
```

Advantages of ADO.NET

1. Fast and efficient data access.
 2. Supports disconnected data handling.
 3. Easy integration with XML.
 4. Better performance and scalability.
 5. Supports multiple databases.
-